Your Automation System, Explained Simply

All seven workflows (CT-01 to CT-07) in one guide, for a complete beginner

Everything here comes from your uploaded files. Examples with made-up values are marked **example**. Nothing was run or tested.

Plain English
In one sentence: when something goes wrong in the business (a late shipment, a payment problem), another system sends a report to this automation. It checks the report, saves it as a case, asks an AI to describe the impact, calculates how urgent it is, asks a human to approve if it is serious, posts an alert in Slack, and keeps a full history. If Slack fails, it retries on its own, and if that fails too, it calls a human.

What is in this guide

#	Section
1	The analogy: a company control room
2	The big picture (two diagrams)
3	The seven workflows at a glance
4	One event, step by step
5	Every node in plain words (all 139 nodes, grouped)
6	How data and workflows connect
7	Decisions, AI, storage and what happens when things fail
8	Three simulated examples
9	Glossary
10	Problems found and how to fix them
11	How to test it
12	One-page summary to teach someone else

1. The analogy: a company control room

Picture a control room where people report business problems. Each person in the room is one part of your system.

Person in the control room	Workflow	What they do
Front desk	CT-01 (start)	Receives the report, checks it is complete, refuses duplicates, opens a case file
Analyst	CT-01 (AI)	Writes a short impact note (an AI does this)
Rules clerk	CT-01 (scoring)	Uses a fixed formula to decide how urgent it is and whether a manager must approve
Manager	CT-01 (approval)	Gets an email, has 45 minutes to click Approve or Reject
File clerk	CT-02	Every change to a case file goes through this clerk, who checks it is allowed and writes it in the logbook
Messenger	CT-03	Posts the alert in Slack
Night watchman	CT-05	Every few minutes: retries failed alerts, finds stuck cases, spots a burst of serious problems
Emergency phone	CT-04	Rings Slack if one of the workflows itself breaks
Reopen desk	CT-06	A person says 'restart case 42'
CRM note-taker	CT-07	Would write the outcome into the customer's HubSpot record (switched off right now)

2. The big picture

Diagram 1: the main journey. Green = good ending, red = stopped or bad ending, yellow = waiting or retrying.

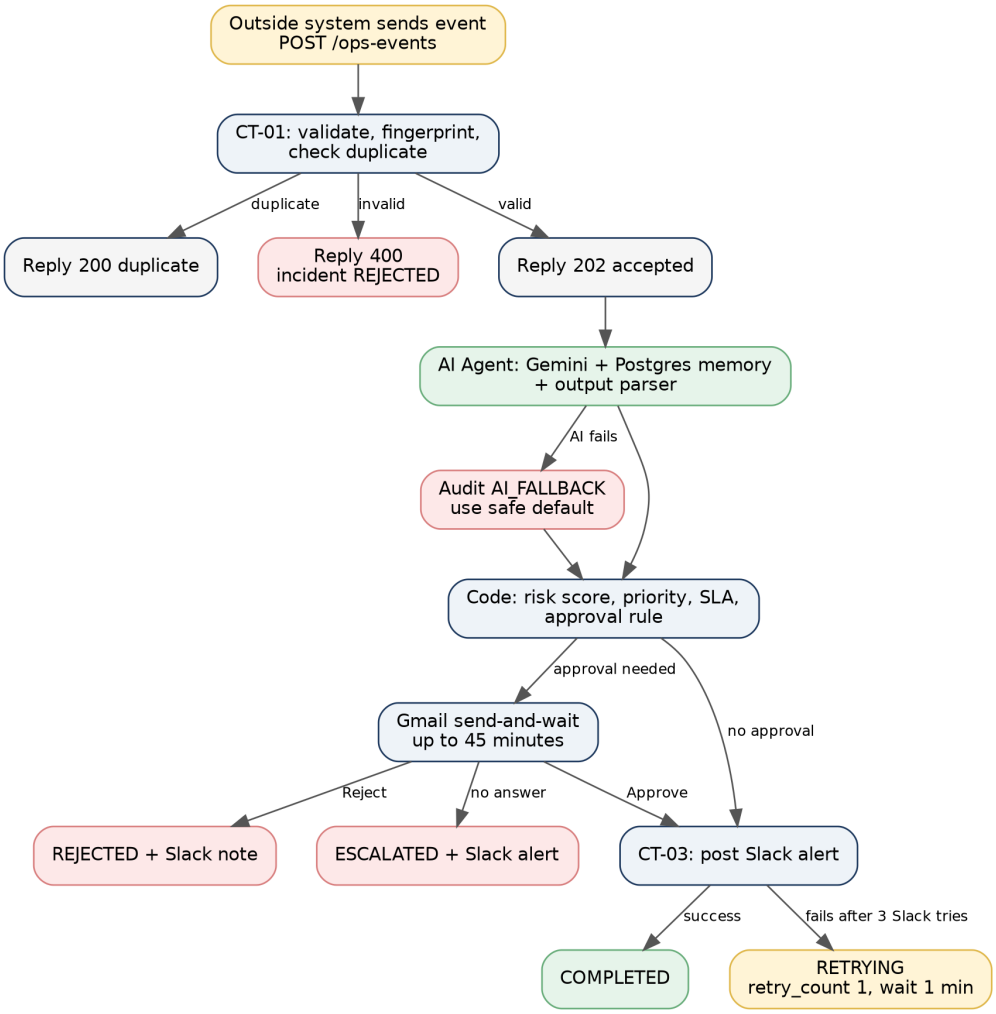
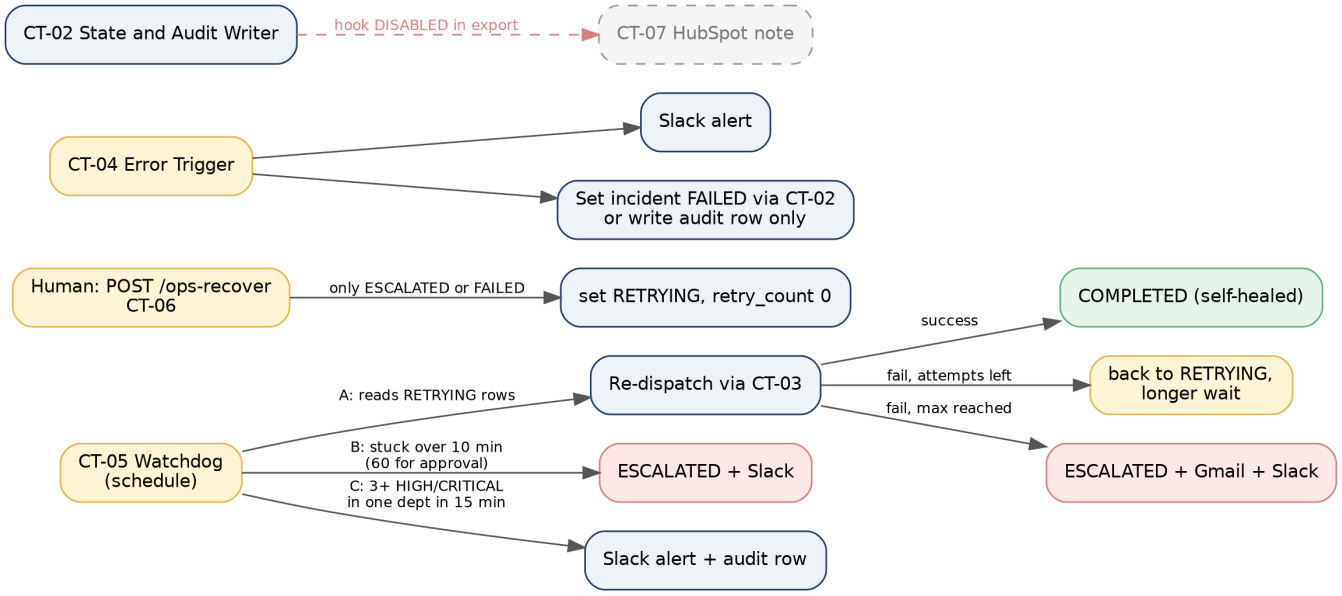


Diagram 2: the helpers. The dashed arrow is a connection that is switched off in your files.



3. The seven workflows at a glance

Workflow	Starts when	Plain-English job	Nodes (working)
CT-01 Main - Event Intake & Orchestration	An outside system sends a report to the web address **/ops-events**	The main pipeline from report to Slack alert	52
CT-02 State & Audit Writer	Another workflow calls it	The only one allowed to change a case's status. Checks the move is legal, saves it, writes the history	9
CT-03 Dispatch Action	CT-01 or CT-05 calls it	Posts the alert to Slack (3 tries inside)	5
CT-04 Global Error Handler	A linked workflow crashes	Alerts Slack and marks the matching case FAILED	8
CT-05 Watchdog	A timer (name says every 5 minutes)	Retries failed alerts, escalates stuck cases, warns about bursts	24
CT-06 Human Recovery Endpoint	A person sends a request to **/ops-recover**	Restarts an ESCALATED or FAILED case	7
CT-07 HubSpot Outcome Sync	CT-02 would call it (currently off)	Adds a note to the customer's HubSpot contact	9

Good to know
A case file is called an **incident** and lives as one row in the database table **incidents**. Its **state** is the stage it is at: RECEIVED, VALIDATING, AI_PROCESSING, CLASSIFIED, AWAITING_APPROVAL, APPROVED, PROCESSING, COMPLETED, REJECTED, RETRYING, ESCALATED or FAILED.

4. One event, step by step

This is the normal route through CT-01, in the order things happen.

#	Step	What happens
1	**Receive**	The webhook **/ops-events** receives a message with event_id, timestamp, department, event_type, severity, entity_id and a payload.
2	**Check**	The message is checked: event_id 3-64 letters/digits, timestamp valid and not in the future, department one of Sales, Logistics, Finance, Customer Service, Inventory, Procurement, event_type in UPPER_SNAKE_CASE, severity LOW/MEDIUM/HIGH/CRITICAL, entity_id present, payload an object.
3	**Fingerprint**	A SHA256 fingerprint of the message is made so it can be identified later.
4	**Duplicate?**	If an incident with the same event_id already exists, the sender gets **200 duplicate** and the workflow stops.
5	**Open the case**	A new incident row is created and logged through CT-02. State becomes VALIDATING.
6	**Valid?**	If the checks failed: reply **400** and state REJECTED. If they passed: reply **202 accepted** immediately (the sender does not wait for the AI).
7	**AI**	State becomes AI_PROCESSING. If the event has a customer email, HubSpot is searched for that contact. The AI Agent then writes an impact assessment. If the AI fails, a safe default is used and 'AI_FALLBACK' is logged.
8	**Score**	A code node (not the AI) computes the risk score, priority (P1-P4), the deadline (SLA) and the list of other departments to notify. State becomes CLASSIFIED.
9	**Approval?**	If the case is serious or the AI is unsure, state becomes AWAITING_APPROVAL and an email asks a human to Approve or Reject, with 45 minutes to answer.
10	**Decision**	Approved: continue. Rejected: state REJECTED and a Slack note. No answer: state ESCALATED and a Slack alert.
11	**Dispatch**	State becomes PROCESSING and CT-03 posts the alert to Slack.
12	**Finish**	Slack worked: COMPLETED. Slack failed after its tries: RETRYING and the Watchdog (CT-05) takes over.

The risk formula

risk score = (severity weight x 15 + AI impact score x 4) x department weight, capped at 100. Severity weights: LOW 1, MEDIUM 2, HIGH 3, CRITICAL 4. Impact is the AI's 1-10 score. Department weights: Finance 1.3, Logistics 1.2, Sales 1.1, Inventory 1.1, Procurement 1.0, Customer Service 1.0.

Risk score	Priority	Deadline (SLA)
80 or more	P1	30 minutes
60-79	P2	2 hours
35-59	P3	8 hours
below 35	P4	24 hours

A human must approve if any one of these is true: severity is HIGH or CRITICAL; risk score is 75 or more; the AI step failed; AI confidence is below 0.60. The AI does not decide this. The code does.

5. Every node in plain words

A **node** is one step in a workflow. Yellow sticky-note nodes (16 in total) only hold comments and do nothing, so they are not listed. **Set** nodes just shape a small package of data. **Code** nodes run a few lines of logic. **Sub-workflow** nodes phone another workflow.

CT-01 Main - Event Intake & Orchestration (52 working nodes)

Stage 1: intake and checking

Node	What it does, simply
Webhook - Receive Ops Event	The front door. Receives the report over the internet (POST /ops-events). The reply is sent later by a Respond node.
Code - Validate Event	Checks every field and tidies the data into a standard shape. Invalid events get a made-up id so they can still be stored.
Crypto - Hash Payload	Makes a SHA256 fingerprint of the original message.
Database - Check Duplicate	Looks in incidents for the same event_id.
IF - Duplicate Event?	Found = duplicate (go to 200 reply). Not found = continue.
Respond - Duplicate (200)	Tells the sender 'already have it' and the workflow ends.
Set - Build Incident Row	Prepares the new case row (fields, the fingerprint, this run's id).
Database - Create Incident	Saves the new incident in Supabase. It gets an id and an incident_no.
Set - Prepare Event Received Log	Prepares a history entry 'EVENT_RECEIVED'.
Sub-workflow - Log Event Received (CT-02)	Asks CT-02 to write that history entry (state does not change).
Set - Prepare VALIDATING Update	Prepares the change RECEIVED to VALIDATING.
Sub-workflow - Set VALIDATING (CT-02)	CT-02 makes the change and logs it.
IF - Event Valid?	Did the checks pass? Yes goes to 202. No goes to 400.
Respond - Invalid (400)	Tells the sender the report was invalid, with reasons.
Set - Prepare REJECTED Update	Prepares VALIDATING to REJECTED.
Sub-workflow - Set REJECTED (CT-02)	CT-02 records the rejection. End of the invalid path.
Respond - Accepted (202)	Tells the sender 'accepted'. The rest runs in the background.

Stage 2: the AI assessment

Node	What it does, simply
Set - Prepare AI_PROCESSING Update	Prepares VALIDATING to AI_PROCESSING.
Sub-workflow - Set AI_PROCESSING (CT-02)	CT-02 records it.
IF - Customer Email Present?	Is payload.customer_email a valid-looking email? Yes goes to HubSpot lookup. No skips it.
HubSpot - Find Contact	Searches HubSpot for that email (read only). If it fails, the workflow carries on without it.

Node	What it does, simply
Code - Build AI Context	Turns the HubSpot answer into a status (NO_EMAIL, FOUND, NOT_FOUND, MULTIPLE_MATCHES, LOOKUP_FAILED), the contact details, and a memory key for the AI.
AI - Assess Impact	The AI Agent. Reads the event and customer context and writes an assessment. Has no tools.
Google Gemini Chat Model	The brain used by the agent (temperature 0.2, so fairly consistent).
Postgres Chat Memory	Gives the agent a memory of earlier conversation for the same memory key.
Structured Output Parser	Forces the AI's reply into a fixed layout (category, impact_score, summary, recommended_action, affected_departments, confidence).
Code - Build AI Fallback Audit	Runs only if the AI fails: prepares a log entry 'AI_FALLBACK'.
Sub-workflow - Log AI Fallback (CT-02)	CT-02 writes that entry, then the flow continues to scoring.

Stage 3: scoring and routing

Node	What it does, simply
Code - Compute Priority & Routing	Uses the AI result (or a safe default if the AI failed), limits its numbers to valid ranges, applies the formula, picks priority and SLA, applies the approval rule, lists departments to notify, and writes the approval email text.
Database - Save Classification	Saves the AI fields, risk score, priority, requires_approval, notify list and SLA on the incident.
Set - Prepare CLASSIFIED Update	Prepares AI_PROCESSING to CLASSIFIED.
Sub-workflow - Set CLASSIFIED (CT-02)	CT-02 records it.
IF - Needs Approval?	Yes goes to the approval stage. No goes straight to PROCESSING.

Stage 4: human approval

Node	What it does, simply
Set - Prepare AWAITING_APPROVAL Update	Prepares CLASSIFIED to AWAITING_APPROVAL.
Sub-workflow - Set AWAITING_APPROVAL (CT-02)	CT-02 records it.
Database - Create Approval Record	Saves a row in the approvals table (status pending).
Gmail - Request Approval	Sends an email with Approve / Reject buttons and waits up to 45 minutes. If the email fails to send, the flow continues and is treated as no answer.
Code - Read Approval Decision	Reads the click: Approved, Rejected, or Timeout (no click). Prepares the matching state change.
Database - Record Decision	Updates the approvals row with the decision.
Set - Prepare Decision Update	Prepares the state change for CT-02.
Sub-workflow - Set Decision State (CT-02)	CT-02 sets APPROVED, REJECTED or ESCALATED.
Switch - Route Decision	Three exits: approved, rejected, timeout.
Slack - Notify Rejection	Posts that a human rejected it. End of the branch.

Node	What it does, simply
Slack - Approval Timeout Alert	Posts that nobody answered in 45 minutes. End of the branch. A human can restart it with CT-06.

Stage 5: dispatch and retry

Node	What it does, simply
Set - Prepare PROCESSING Update	Prepares the change to PROCESSING (from CLASSIFIED or APPROVED).
Sub-workflow - Set PROCESSING (CT-02)	CT-02 records it.
Set - Prepare Dispatch	Packs { incident, attempt: 1 } for CT-03.
Sub-workflow - Dispatch Action (CT-03)	Calls CT-03 and waits. Two exits: success or error.
Set - Prepare COMPLETED Update	On success: prepares PROCESSING to COMPLETED.
Sub-workflow - Set COMPLETED (CT-02)	CT-02 records COMPLETED and the finish time.
Code - Build Retry Update	On failure: prepares PROCESSING to RETRYING with retry_count 1 and a retry time 1 minute later (the code comment says 'demo value; use 5 in production').
Sub-workflow - Set RETRYING (CT-02)	CT-02 records RETRYING. CT-05 takes over from here.

CT-02 State & Audit Writer (9 working nodes)

Node	What it does, simply
Trigger - Receive State Change	The phone that rings when another workflow calls. Accepts any data.
Code - Validate Transition	The rule book. Refuses missing data and illegal moves (for example COMPLETED back to PROCESSING). Prepares three packages: update, audit row, receipt.
Set - Row: Incident Update	Keeps only the update package.
Database - Update Incident	Changes the incident's state and any extra fields in the incidents table.
Set - Row: Audit Log	Fetches the audit package again.
Database - Write Audit Log	Adds one row to audit_logs: what happened, from which state to which, by whom.
IF - Final State Reached?	Is the new state COMPLETED, REJECTED, ESCALATED or FAILED?
Sub-workflow - Sync Outcome to HubSpot (CT-07)	Would start CT-07 without waiting. **Disabled** in your export, so it does nothing.
Set - Return Result	Sends back the receipt { ok, incident_id, new_state, context }.

CT-03 Dispatch Action (5 working nodes)

Node	What it does, simply
Trigger - Receive Dispatch Request	The phone that rings. Expects { incident, attempt }.
Database - Get Dispatch Config	Reads two settings from system_config: dispatch_channel_id and simulate_slack_failure.

Node	What it does, simply
Code - Build Dispatch Message	Builds the Slack text (priority, INC number, department, event, entity, severity, risk, AI summary, recommended action, who to notify, deadline, attempt number). If simulate_slack_failure is true it uses a fake channel so Slack will refuse (the failure demo).
Slack - Post Incident Alert	Posts to the channel. Tries up to 3 times, 2 seconds apart. If all fail, CT-03 fails on purpose so the caller can retry later.
Set - Return Result	Replies { ok: true, incident }.

CT-04 Global Error Handler (8 working nodes)

Node	What it does, simply
Error Trigger - Workflow Failed	Starts when a workflow linked to it crashes (only CT-06 is linked in your files).
Slack - Send Error Alert	Posts which workflow failed, the last node, the error and a link. If Slack fails, the rest still runs.
Database - Find Incident by Execution	Looks for the incident created by the crashed run.
IF - Incident Found and Open?	Found and not already finished (COMPLETED, REJECTED, FAILED, ESCALATED)?
Set - Prepare FAILED Update	Yes: prepares a change to FAILED with the error text.
Sub-workflow - Set FAILED (CT-02)	CT-02 sets FAILED and logs it.
Set - Build Error Audit Row	No: prepares a history row with just the error.
Database - Log Error Only	Saves that row in audit_logs. The state is not changed.

CT-05 Watchdog (24 working nodes)

It starts three separate jobs at the same moment.

Node	What it does, simply
Schedule - Every 5 Minutes	The timer. (The exact number of minutes is not stored in the export.)
Job A: retry failed alerts	
Database - Get Retrying Incidents	Gets all incidents in RETRYING.
Code - Select Due Retries	Keeps only those whose retry time has arrived.
Loop - Each Due Retry	Takes them one at a time.
Set - Prepare PROCESSING Update	Prepares RETRYING to PROCESSING.
Sub-workflow - Set PROCESSING (CT-02)	CT-02 records it.
Set - Prepare Re-Dispatch	Packs { incident, attempt = retry_count + 1 }.
Sub-workflow - Re-Dispatch Action (CT-03)	Tries Slack again. Success or error exit.
Set - Prepare Self-Healed Update	Success: prepares PROCESSING to COMPLETED.
Sub-workflow - Set COMPLETED (CT-02)	CT-02 records it, then the loop takes the next one.

Node	What it does, simply
Code - Build Failure Update	Failure: adds 1 to the count. If it reaches max (the row's max_retries, or 3), the new state is ESCALATED. Otherwise RETRYING with a longer wait (attempts x 1 minute).
Sub-workflow - Record Failure (CT-02)	CT-02 records RETRYING or ESCALATED.
IF - Retries Exhausted?	Escalated? Yes goes to email and Slack. No goes back to the loop.
Gmail - Send Escalation Email	Emails a person that retries ran out and how to recover.
Slack - Send Escalation Alert	Posts the escalation in Slack, then the loop continues.
Job B: find stuck cases	
Database - Get Open Incidents	Gets incidents that are not finished.
Code - Find Stale Incidents	No update for more than 10 minutes (60 for AWAITING_APPROVAL) in RECEIVED, VALIDATING, AI_PROCESSING, CLASSIFIED, APPROVED, PROCESSING means stuck.
Sub-workflow - Set ESCALATED (CT-02)	CT-02 sets ESCALATED.
Slack - Send Stale Alert	Posts the stuck-case alert.
Job C: spot a burst	
Database - Get Recent Incidents	Gets incidents from the last 15 minutes.
Code - Detect Systemic Failures	3 or more HIGH/CRITICAL in one department = burst. Remembers it already alerted that department for 30 minutes.
Slack - Send Systemic Alert	Posts the burst warning.
Set - Build Audit Row	Prepares a history row.
Database - Log Systemic Alert	Saves it in audit_logs.

CT-06 Human Recovery Endpoint (7 working nodes)

Node	What it does, simply
Webhook - Receive Recovery Request	Receives POST /ops-recover with incident_no, and optionally actor and note.
Database - Get Incident	Finds the incident by incident_no. Even if nothing is found, one empty item continues.
IF - Incident Recoverable?	Is the state ESCALATED or FAILED?
Set - Prepare RETRYING Update	Yes: prepares a change to RETRYING with retry_count 0 and retry time now.
Sub-workflow - Set RETRYING (CT-02)	CT-02 records it, with the person's actor and note.
Respond - Recovered (200)	Replies 'recovering. Watchdog will retry on its next tick'.
Respond - Not Recoverable (409)	No: replies 409 with the current state (or NOT_FOUND). Nothing changes.

CT-07 HubSpot Outcome Sync (9 working nodes, not running today)

Node	What it does, simply
Trigger - Receive Final State	Starts when called with an incident_id.

Node	What it does, simply
Database - Get Incident	Reads the incident.
IF - Customer Email Present?	No valid email: stop. Nothing to do.
HubSpot - Find Contact	Searches HubSpot by email. Never creates contacts.
IF - Contact Found?	Found goes to the note. Not found goes to the audit row.
Code - Build Note Request	Writes the note text: incident number, priority, outcome, department, event type, AI summary, last error.
HubSpot - Create Note	Creates the note and attaches it to the contact.
Code - Build Audit Row	Chooses HUBSPOT_NOTE_CREATED, HUBSPOT_CONTACT_NOT_FOUND or HUBSPOT_SYNC_FAILED.
Database - Log HubSpot Sync	Saves that row in audit_logs.

6. How data and workflows connect

6.1 Data moves in 'envelopes'

Each step passes an **item** (an envelope of labelled facts) to the next. Labels are called **field names**, such as `incident_id` or `severity`. The notation `{{ $json.severity }}` means 'read the label severity from the envelope in front of me'. Some nodes keep the envelope and add to it; some replace it (the Set nodes in raw mode); some throw it away. That is why later nodes often reach back to an earlier node by name, like `$('Database - Create Incident').first().json` ('go back to that step and read what it produced').

The request every state change uses (sent to CT-02)

Field	Meaning
incident_id, event_id	Which case
previous_state, new_state	Where it is now and where it should go
event, status, error_message	The history entry: what happened, SUCCESS/WARNING/FAILURE/INFO, and any error
actor	Who did it (system, system:watchdog, a person's email)
patch	Extra columns to change (retry_count, next_retry_at, last_error ...)
context	Anything to be echoed back in the receipt

6.2 How the workflows talk

Link	Type	Detail
CT-01 to CT-02 (11 nodes)	Direct call, waits	Every state change and history entry. A failure inside CT-02 fails CT-01.
CT-01 to CT-03	Direct call, waits	Dispatch. Has an error exit: failure leads to RETRYING.
CT-05 to CT-02 (4 nodes), CT-03 (1)	Direct calls, wait	Retry, completion, escalation, stuck cases.
CT-04 to CT-02	Direct call, waits	Sets FAILED.
CT-06 to CT-02	Direct call, waits	Sets RETRYING.
CT-02 to CT-07	Direct call, does NOT wait	Disabled in the export.
CT-01 and CT-06 to CT-05	**Through the database**	They set RETRYING and next_retry_at. CT-05 reads the incidents table later. No direct call.
CT-06 to CT-04	**n8n setting**	CT-06's Error Workflow setting points at CT-04. CT-01 and CT-05 have no such setting.

7. Decisions, AI, storage and failures

7.1 Every decision point

Where	Question	Yes / No
CT-01 IF - Duplicate Event?	Same event_id already stored?	Yes: reply 200, stop. No: create incident.
CT-01 IF - Event Valid?	Did all checks pass?	Yes: reply 202. No: reply 400, REJECTED.
CT-01 IF - Customer Email Present?	Valid-looking customer email?	Yes: look up HubSpot. No: skip.
CT-01 AI - Assess Impact (two exits)	Did the AI work?	Yes: score. No: log fallback, then score with safe defaults.
CT-01 IF - Needs Approval?	Approval rule met?	Yes: ask a human. No: dispatch.
CT-01 Switch - Route Decision	What did the human do?	Approved: dispatch. Rejected: Slack note. No answer: Slack alert (ESCALATED).
CT-01 Sub-workflow - Dispatch (two exits)	Did Slack post work?	Yes: COMPLETED. No: RETRYING.
CT-02 IF - Final State Reached?	COMPLETED, REJECTED, ESCALATED or FAILED?	Yes: (would) sync to HubSpot. No: return receipt.
CT-04 IF - Incident Found and Open?	Linked and not finished?	Yes: set FAILED. No: audit row only.
CT-05 Code - Select Due Retries	Is the retry time reached?	Yes: retry now. No: leave.
CT-05 Re-Dispatch (two exits)	Slack worked?	Yes: COMPLETED. No: count the attempt.
CT-05 IF - Retries Exhausted?	Reached max attempts?	Yes: ESCALATED, email, Slack. No: RETRYING.
CT-06 IF - Incident Recoverable?	ESCALATED or FAILED?	Yes: RETRYING, reply 200. No: reply 409.
CT-07 IF nodes	Email valid? Contact found?	Described under CT-07.

7.2 The AI, simply

- **An AI model call** answers one request. An **AI Agent** can also use tools and memory. Your agent has memory (Postgres) and an output parser but **no tools**, so it can only write an assessment. It cannot send, change or approve anything.
- **It receives** the cleaned event and the HubSpot status and contact details (if a customer email was found).
- **It returns** a category, an impact score 1-10, a summary, a recommended action, affected departments and a confidence 0-1, forced into shape by the parser.
- **The code never fully trusts it.** It limits the numbers to valid ranges, uses its own formula for risk and priority, and decides approval itself. If the AI fails or has low confidence, a human must approve.
- **AI can still be wrong.** The human approval step is the safety net for serious cases. Routine cases (low severity, high confidence) go straight to Slack on the AI's word.

7.3 Where information is stored

Place	What is in it	Used by
Supabase table **incidents**	The current state of every case plus AI results, scores, retry info	All workflows
Supabase table **audit_logs**	The history: one row per event. Rows are only added.	CT-02 (and CT-04, CT-05, CT-07 directly)
Supabase table **approvals**	Approval requests and decisions	CT-01
Supabase table **system_config**	Slack channel id and the failure-demo switch	CT-03
Postgres (AI memory)	The AI agent's conversation memory	CT-01
HubSpot	Notes on contacts (CT-07, currently off). Contacts are only read, never created.	CT-01 read, CT-07 write

Good to know
Data passed between nodes disappears when the run ends. Only database writes (and HubSpot notes) are kept.

7.4 When things go wrong: five safety nets

#	Safety net	What it covers
1	Slack retries (CT-03)	3 tries, 2 seconds apart, for a temporary Slack problem
2	RETRYING and the Watchdog (CT-05)	Up to 3 dispatch attempts in total, with growing waits, then ESCALATED plus email and Slack
3	Stuck-case scan (CT-05)	A case with no progress for 10 minutes (60 for approvals) is ESCALATED
4	Error handler (CT-04)	Alerts Slack and marks the incident FAILED if a **linked** workflow crashes. Linked today: CT-06 only
5	Human recovery (CT-06)	A person restarts an ESCALATED or FAILED case

Two kinds of error handling exist: **inside a workflow** (error exits, retries, IF nodes) and an **Error Trigger workflow** (CT-04), which n8n starts only when another workflow's run crashes and is linked to it.

8. Three simulated examples

These are simulations with made-up values, not real runs.

Example A: a routine event, no approval

```
Sent to /ops-events:
{ "event_id": "EVT-2001", "timestamp": "2026-10-10T08:00:00Z", "department": "Sales",
  "event_type": "PRICE_QUERY", "severity": "LOW", "entity_id": "QUO-778",
  "payload": { "customer_email": "buyer@example.com" } }
```

- Validated, no duplicate, incident INC-n created. Reply **202**. State: VALIDATING, then AI_PROCESSING.
- HubSpot is searched for buyer@example.com. The AI answers: impact 3, confidence 0.9.
- Risk = (1 x 15 + 3 x 4) x 1.1 = 29.7, so **30**. Priority **P4**, deadline 24 hours.
- No approval rule met (LOW, risk under 75, AI worked, confidence 0.9). State: CLASSIFIED, PROCESSING.
- CT-03 posts to Slack. State: **COMPLETED**. The audit_logs table has about 6 rows for this incident.

Example B: a serious event that needs approval

Department Logistics, event_type SHIPMENT_DELAY, severity **HIGH**, AI impact 7.

- Risk = (3 x 15 + 7 x 4) x 1.2 = 87.6, so **88**. Priority **P1**, deadline 30 minutes. Notify list starts with Customer Service, Sales, Finance.
- Approval needed (severity HIGH, and risk 88 is 75 or more). State: AWAITING_APPROVAL. A row appears in approvals and an email is sent.
- **If the approver clicks Approve:** APPROVED, PROCESSING, Slack alert, COMPLETED.
- **If Reject:** REJECTED and a Slack note. **If no click in 45 minutes:** ESCALATED and a Slack alert; a person can later restart it with CT-06.

Example C: Slack is down

When	What happens	State
Attempt 1 (CT-01)	3 Slack tries fail. CT-03 fails, CT-01 takes the error exit	RETRYING, retry_count 1, retry in 1 minute
Attempt 2 (CT-05)	Watchdog picks it up, sets PROCESSING, calls CT-03, fails again	RETRYING, retry_count 2, retry in 2 minutes
Attempt 3 (CT-05)	Fails again. 3 attempts reached	**ESCALATED**, escalation email and Slack alert
Later	A person fixes Slack and posts to /ops-recover with the incident_no	RETRYING, count 0, then the next Watchdog pass retries

Demo: set simulate_slack_failure = true in system_config to cause this. Setting it back to false before the next Watchdog pass lets the retry succeed ('self-healing').

9. Glossary

Term	Meaning
Workflow	A saved automation made of connected steps.
Node	One step in a workflow.
Trigger	The node that starts a workflow.

Term	Meaning
Execution	One run of a workflow.
Item / JSON	An envelope of labelled facts (label: value). JSON is the way it is written.
Field	One labelled fact, such as severity.
Expression	A small instruction inside {{ }} that reads data while the workflow runs.
Webhook	A web address that lets another system send information into a workflow.
API	A way for programs to talk to each other, like Slack or HubSpot.
Sub-workflow	A workflow called by another one, like a phone call.
Payload	The extra details inside a message.
State / Status	The stage an incident is at.
Audit log	The history book: one row per event, never edited.
Database / table	A digital filing cabinet / one drawer in it.
Supabase	The database service used here.
CRM / HubSpot	Software that stores customer records.
AI Agent	An AI that can also use tools and memory (yours has memory, no tools).
Retry	Trying a failed action again.
Escalation	Passing a problem to a human.
Error Trigger	A special start node that runs when another workflow crashes.
SLA	The deadline to deal with an incident.
Credentials	Saved logins that let n8n use Slack, Gmail, etc.

10. Problems found and how to fix them

Confirmed = visible directly in the JSON. **Possible** = a risk that needs a live test or knowledge of things outside the files.

#	Problem	Why it matters	Suggested fix	Type
1	CT-01 and CT-05 have no Error Workflow set. Only CT-06 points to CT-04.	If CT-01 or CT-05 crashes there is no Slack alert and no automatic FAILED.	In CT-01 and CT-05 settings choose CT-04 as Error Workflow.	Confirmed
2	The CT-02 node that starts CT-07 is disabled.	No HubSpot notes are ever written.	Enable it, then test that CT-07 receives incident_id.	Confirmed
3	The AI's instructions ask for a different output than the parser enforces (13 fields and a one-word category versus 6 fields and a fixed category list).	The AI may fail the parser and drop into the fallback, forcing human approval more often.	Make the prompt and the parser describe the same fields.	Confirmed
4	Neither webhook has authentication.	Anyone with the URL can create incidents or restart incidents.	Add header or basic authentication, or protect it upstream.	Confirmed
5	The approval email goes to oluwateey12@example.com but the log records the approver as your.email@example.com (a placeholder).	The audit trail names the wrong person. Both look like placeholder addresses.	Use one real address, ideally from one setting.	Confirmed
6	CT-02 checks the move against the previous_state the caller claims, not the stored state, and its two database writes are not tied together.	An out-of-date caller could write an illegal move; a failure between writes leaves no history row.	Read the current state in CT-02 and reject mismatches.	Possible
7	Duplicates are detected by event_id only. The fingerprint is stored but not used for this.	Same id with different content is treated as a duplicate. Two identical requests at the same instant might both get through.	Compare the fingerprint too; make event_id unique in the database.	Possible
8	A missing Slack channel setting in CT-03 looks like a Slack failure.	Pointless retries, then escalation.	Handle the config error separately and alert immediately.	Confirmed
9	Restarting an approval-timeout incident through CT-06 leads to dispatch with no approval.	A serious case can be sent without a human having approved it.	On recovery, send such cases back to AWAITING_APPROVAL, or require a named approver.	Possible
10	Customer email and CRM context are sent to the AI and kept in its memory and in the incident payload.	Personal data is stored in more places.	Decide what may be stored and for how long.	Possible
11	Retry delay is 1 minute ('demo value; use 5 in production') and the timer minutes are not stored in the export.	Behaviour in production may differ from what you expect.	Check both before going live.	Confirmed / cannot confirm

Good to know
Nothing here was changed in your workflows.

11. How to test it

Use made-up data only. In n8n open **Executions**, click a run, then click any node to see its **Input** and **Output**. Check the Supabase tables incidents and audit_logs after each test. I have not run any of these, so I cannot say the results.

Test	Send / do	Expect	Failure looks like
1 Invalid	POST /ops-events with severity 'URGENT'	Reply 400 listing errors; incident REJECTED; the AI nodes do not run	Reply 202, or no incident row
2 Duplicate	Send the same event_id twice	Second reply 200 duplicate; only one incident row	Two rows
3 Routine	LOW severity, department Sales	202, then COMPLETED; no approval email; Slack alert; several audit rows	Stuck in a middle state
4 Needs approval	HIGH severity, Logistics	AWAITING_APPROVAL, an email, a row in approvals; Approve leads to COMPLETED	No email, or no approvals row
5 Timeout	Do not click for 45 minutes	ESCALATED and the timeout Slack alert	Waits forever
6 Slack failure	Set simulate_slack_failure = true, send a routine event; later set it to false	RETRYING, then COMPLETED after the flag is false; audit shows DISPATCH_FAILED then success	No retry happens
7 Escalation	Leave the flag true	After 3 attempts: ESCALATED, escalation email and Slack alert	Keeps retrying
8 Recovery	POST /ops-recover with that incident_no, then with a COMPLETED one	200 recovering; the COMPLETED one gives 409	Both accepted
9 Error handler	Make CT-06 fail on purpose (for example bad credentials) in a test copy	Slack error alert from CT-04. Repeat on CT-01 to see whether it fires (finding 1)	No alert

12. One-page summary you can teach from

What it is. A control room that turns a business problem report into a tracked incident, a Slack alert and a full history, with automatic retries and human safety nets.

The seven parts. CT-01 is the main pipeline. **CT-02** is the file clerk that changes status and writes the history. **CT-03** posts to Slack. **CT-05** is the night watchman. **CT-06** lets a person restart a case. **CT-04** alerts if a workflow crashes. **CT-07** would add HubSpot notes (off).

The journey.

```
Report arrives -> checked -> duplicate? -> case opened -> quick reply (202)
  -> AI writes impact note -> code scores risk and priority
  -> serious? -> email a human (45 min) -> approve / reject / no answer
  -> Slack alert -> COMPLETED
  -> Slack fails? -> RETRYING -> Watchdog retries (3 attempts) -> ESCALATED -> human restarts with CT-06
```

Three rules to remember.

- **The AI advises, the code decides.** The AI writes the assessment. Priority, deadline and approval come from fixed rules.
- **Only CT-02 changes a status**, and it always writes the history.
- **The database is the shared memory.** CT-05 finds work simply by reading the incidents table.

Where it is weak today. CT-04 is only attached to CT-06; the HubSpot step is switched off; the AI prompt and parser disagree; the webhooks are open; two email addresses look like placeholders.

How to explain it in 20 seconds. 'Something goes wrong, a report comes in, the system checks it, asks an AI what it means, scores how urgent it is, asks a manager if it is serious, tells the team on Slack, and writes everything down. If Slack is down it keeps trying and finally calls a person.'